Now, let us understand how the program works. The prime number p = 23 and the primitive root g = 5 act as public parameters. Alice selects her private key a = 6 and computes her public value $A = g^a \bmod p$, while Bob selects his private key b = 15 and computes his public value $B = g^b \bmod p$. They exchange A and B over an insecure channel. Alice then calculates the shared secret $S\_alice = B^a \bmod p$, and Bob calculates $S\_bob = A^b \bmod p$. Both arrive at the same shared secret, which can be used as a common key for secure communication. The security of the key exchange lies in the fact that neither Alice's nor Bob's secret key is ever transmitted over the insecure channel.

Upon executing the program 'toydh38.sage', the output shown in Figure 3 is obtained. The figure clearly shows that both Alice and Bob arrive at the same secret key, confirming the successful execution of the Diffie–Hellman key exchange.

Now, let us implement a more realistic version of the Diffie–Hellman key exchange that is practically secure. The SageMath program named 'dh39.sage' shown below demonstrates a simplified yet powerful implementation capable of generating and exchanging keys that are more than 150 digits long.

```
p = random_prime(2^512, lbound=2^511)
g = primitive_root(p)
a = ZZ.random_element(2, p-1)
A = power_mod(g, a, p)
b = ZZ.random_element(2, p-1)
B = power_mod(g, b, p)
S_alice = power_mod(B, a, p)
S_bob   = power_mod(A, b, p)
print("Large prime p:", p)
print("Generator g:", g)
print("Shared secret (Alice):", S_alice)
print("Shared secret (Bob):  ", S_bob)
print("Keys match? ", S_alice == S_bob)
```

Now, let us understand how the program works. It first generates a random 512-bit prime number p and a primitive root g modulo p. Alice and Bob each select their private keys a and b randomly between 2 and p-1, then compute their public values $A = g^a \bmod p$ and $B = g^b \bmod p$. They exchange these public values and use them to compute the shared secret key: Alice calculates $S\_alice = B^a \bmod p$ and Bob calculates $S\_bob = A^b \bmod p$. The program prints the large prime p, the generator g, both computed shared secrets, and confirms that the keys match, demonstrating a successful secure key exchange.

I was unable to run the program 'dh39.sage' locally on my computer due to the intensive computations involved. Therefore, I relied on CoCalc to execute the program, which produced the results effortlessly. Recall that we introduced

```
# sage toydh38.sage
Public parameters: p = 23 , g = 5
Alice sends A = 8
Bob sends B = 19
Alice computes secret: 2
Bob computes secret:   2
```

Figure 3: Toy DH key exchange

```
Large prime p:
11822411686310179836901213321689995574678059898499810593898774912448430647346383694034199985816434513941337434638889352677491765257249026562728228696145419
Generator g: 2
Shared secret (Alice):
18602081059581255753265263280285449978460874771568079253635865437336891220106895707260074290121164659804319670072512089357779900197347919310045339240721 31
Shared secret (Bob):
18602081059581255753265263280285449978460874771568079253635865437336891220106895707260074290121164659804319670072512089357779900197347919310045339240721 31
Keys match?  True
```

Figure 4: Diffie–Hellman key generation

CoCalc at the beginning of this series, so I will not go over the execution steps again. Unless you have access to a very powerful system, it is advisable to run this program on CoCalc for convenience and efficiency.

Upon executing the program 'dh39.sage' on CoCalc, the output shown in Figure 4 is obtained. The figure clearly shows that the keys computed by both Alice and Bob are identical. It can also be observed that the shared key is more than 150 digits long, corresponding to a 512-bit key. Although a 512-bit key is no longer considered secure against modern computational power, the same program can easily be modified to generate much larger keys, thereby enhancing security.

Now it is time to wrap up our discussion—but our journey with public-key cryptography (PKC) is far from over. Consider the following scenario: you receive an email from a friend who promises to lend you ₹1 million. Later, your friend insists that he never sent such a message and that someone must have spoofed his email address. How can you verify that the email truly came from your friend? Clearly, there must be a way to ensure the authenticity and integrity of digital communication. This is where digital signatures come into play. Once again, PKC provides the foundation—enabling us to sign documents and messages securely in the digital world.

In the next article in this series, we will explore digital signatures, along with the related concepts of hashing and message authentication. END

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He is currently working as an assistant professor (senior scale) at Manipal Institute of Technology, Bengaluru.